Week 2: Directed Acyclic Graphs (DAGs)

Davood Tofighi, Ph.D.

R Packages Required

```r
pacman::p_load(
  "dagitty",
  "ggplot2",
  "ggdag",
  "tidyverse",
  "bnlearn",
  "Rgraphviz",
  "ggpubr"
)

ggplot2::theme_set(ggpubr::theme_pubr())
set.seed(313)
```

Welcome back, class. In our previous sessions, we've built a solid foundation in the language of potential outcomes. We now understand that our primary goal is to estimate the Average Treatment Effect (ATE) and that to do so from observational data, we must assume **conditional exchangeability**. This assumption, however, can feel abstract.

Today, we dive deeper into **Directed Acyclic Graphs (DAGs)**, a framework that makes these abstract assumptions concrete and visual. DAGs are the single most important tool for thinking clearly about the structure of confounding, selection bias, and causation. Mastering them will allow you to diagnose bias in research designs and to develop a principled strategy for statistical adjustment.

Today's Learning Objectives

1. **Master** the components of a DAG and the profound causal assumptions they represent.

2. **Analyze** the flow of association through chains, forks, and colliders, using multiple real-world examples and numerical demonstrations.
3. **Define and rigorously apply** the rules of **d-separation** to test for conditional independence in complex causal systems.
4. **Translate** confounding into the graphical concept of a **backdoor path** and use the **Backdoor Criterion** to identify valid adjustment sets.
5. **Critically evaluate** different adjustment strategies, understanding why "adjusting for everything" is a dangerous approach.
6. **Introduce** the **Front-Door Criterion** as an alternative strategy for identification when unmeasured confounding is present.

1. The Anatomy of a Causal Graph: From Pictures to Assumptions

A **Directed Acyclic Graph (DAG)** is a mathematical object we use to draw our scientific beliefs. It's composed of nodes (variables) and edges (arrows) with one critical rule.

- **Nodes (Vertices)**: These represent all relevant variables in a system: the exposure/treatment, the outcome, and any covariates.
- **Directed Edges (Arrows)**: An arrow from A to B ($A \to B$) asserts a **direct causal relationship**. The key word is **direct**; A is assumed to have a causal influence on B that is not mediated by any other variable present in the graph. In this relationship, A is a **parent** of B.
- **Acyclicity**: There are no directed paths that start at a node and circle back to it. This means a variable cannot be its own ancestor. If you believe a feedback loop exists (e.g., poverty causes poor health, which in turn worsens poverty), a standard DAG is not the right tool. You would need more advanced methods, such as those for dynamic treatment regimes or marginal structural models, which we may touch on later in the course.

What DAGs Assume: The Holy Trinity of Causal Graph Assumptions

When you draw a DAG, you are implicitly making three profound assumptions about the world.

1. **The Causal Markov Assumption**: This is the big one. It states that conditional on its direct parents, any node is independent of its non-descendants. In plain English, the arrows pointing directly into a variable capture **all** of its direct causes within the system. An absent arrow is a strong claim of no direct causal effect.
2. **The Faithfulness Assumption**: This assumes that all statistical dependencies present in the data are represented by a path in the graph. For example, if a drug has two effects that perfectly cancel each other out, it would appear independent of the outcome in the data, violating faithfulness. We generally assume faithfulness holds.

3. **SUTVA (Stable Unit Treatment Value Assumption)**: Just as with potential outcomes, we must assume that one person's treatment doesn't affect another's outcome (no interference) and that the treatment is well-defined (consistency). A DAG doesn't explicitly show this, but it's a background assumption for the whole enterprise.

2. The Three Channels of Association: A Deeper Look

Association is a statistical property that flows through a graph like water through pipes. To isolate the causal flow, we must learn to identify and block the non-causal flows. All paths are made of three elementary building blocks.

Building Block 1: The Chain (Mediation)

$$A \to M \to Y$$

This structure represents a causal process. It's how things work. Association flows from A to Y *through* M.

- **Default Status**: **OPEN**. This path transmits (causal) association.
- **Conditioning on M**: **BLOCKS** the path.
- **Examples**:
  - **Public Health**: A public awareness campaign (A) increases sunscreen use (M), which in turn lowers the rate of skin cancer (Y).
  - **Social Science**: Receiving food stamps (A) reduces a family's financial stress (M), which in turn improves children's educational outcomes (Y).
- **Numerical Demonstration**: Imagine A is a binary treatment, and its effect on Y is entirely mediated through M. Let the true causal model be: $M = 2A + \epsilon_M$; $Y = 3M + \epsilon_Y$ Substituting the first equation into the second gives $Y = 3(2A + \epsilon_M) + \epsilon_Y = 6A + \text{noise}$. The true total causal effect of A on Y is 6.

As you can see, the model that correctly omits the mediator finds the true total effect of ~6. The model that adjusts for the mediator M finds that A has no remaining effect on Y because it has blocked the only causal pathway.

Table 1: Direct and Indirect Effects

```
n <- 1000
A <- rbinom(n, 1, 0.5)
M <- 2 * A + rnorm(n)
Y <- 3 * M + rnorm(n)

# Model without adjusting for the mediator M
total_effect_model <- lm(Y ~ A)
cat("Total Effect of A on Y:", coef(total_effect_model)["A"]) # Should be ~6
```

```
Total Effect of A on Y: 6.012541
```

```
# Model *incorrectly* adjusting for mediator M
direct_effect_model <- lm(Y ~ A + M)
cat(
  "\nEffect of A on Y when controlling for M:",
  coef(direct_effect_model)["A"]
) # Should be ~0
```

```
Effect of A on Y when controlling for M: -0.1288009
```

Building Block 2: The Fork (Confounding)

$$A \leftarrow X \rightarrow Y \tag{1}$$

This is the graphical signature of confounding. [cite_start]A common cause, X, creates a non-causal association between A and Y[cite: 72, 74].

- **Default Status**: **OPEN**. This path transmits non-causal association. This is a **backdoor path**.
- **Conditioning on X**: **BLOCKS** the path.
- **Examples**:
  - **Medicine**: Patients with a severe prognosis (X) are more likely to choose an aggressive surgery (A) and are also more likely to have a poor outcome (Y).
  - **Finance**: A country's underlying economic stability (X) influences both its decision to adopt a specific monetary policy (A) and its future GDP growth (Y).
- **Goal**: To get the causal effect, you **must** condition on X to block this spurious path. This is the graphical analog of achieving conditional exchangeability.

Building Block 3: The Collider

$$A \to C \leftarrow Y \tag{2}$$

This is the most counter-intuitive structure. Two independent causes converge on a common effect.

- **Default Status**: **BLOCKED**. No association flows through a collider.
- [cite_start]**Conditioning on C**: **OPENS** the path, creating spurious association[cite: 107, 132, 139].
- **Examples**:
  - **Hiring**: A company hires a new programmer ($C = 1$) based on their coding skill (A) or their communication skill (Y). Among those hired (conditioning on C), someone with poor coding skills is more likely to have excellent communication skills.
  - **Disease Symptoms**: A cough (C) can be caused by the flu (A) or by allergies (Y). If you meet someone who is coughing, and you learn they do *not* have the flu, you immediately believe it's more likely they have allergies. Knowing the status of one cause gives you information about the other, but only because you've conditioned on their common effect (the cough).

3. D-Separation: A Masterclass with a Complex Graph

D-separation is the complete algorithm for determining conditional independence in a DAG. Let's apply it to a complex case to solidify our understanding.

```
# A complex DAG for our d-separation quiz
complex_dag <- dagitty(
  'dag{
  A [exposure]
  Y [outcome]
  V; B; C; D; E; F

  V -> A; B -> V; B -> D
  D -> A; D -> E
  C -> A; C -> Y
  E -> Y
  A -> F -> Y
}'
)
ggdag(complex_dag, layout = "nicely") + theme_dag_blank()
```

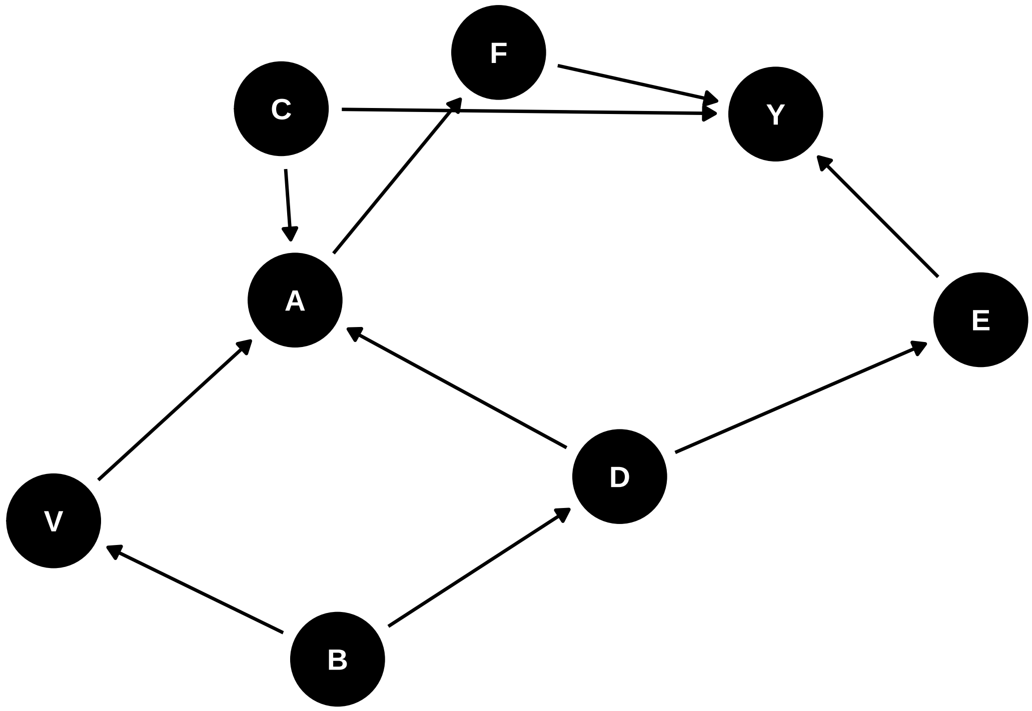


Figure 1: A Complex DAG for our d-separation quiz

Let's trace the paths for a few queries.

Query 1: Find all paths between A and Y.

- **Path 1**: $A \to F \to Y$ (Causal Chain)
- **Path 2**: $A \leftarrow C \to Y$ (Backdoor Fork)
- **Path 3**: $A \leftarrow V \leftarrow B \to D \to E \to Y$ (Backdoor Chain/Fork Combo)
- **Path 4**: $A \leftarrow D \to E \to Y$ (Backdoor Fork/Chain Combo)

Query 2: Is A independent of Y given no conditioning? $(A \perp\!\!\!\perp Y)$

- **Path 1** $(A \to F \to Y)$: Open (chain).
- **Path 2** $(A \leftarrow C \to Y)$: Open (fork).
- **Path 3** $(A \leftarrow V \leftarrow B \to D \to E \to Y)$: Open (chain/fork).
- **Path 4** $(A \leftarrow D \to E \to Y)$: Open (fork/chain).
- **Answer**: No. Multiple open paths transmit association.

Query 3: Is A independent of Y given {C, D}? $(A \perp\!\!\!\perp Y|C, D)$

- **Path 1** $(A \to F \to Y)$: Still **open**. C and D are not on this path.
- **Path 2** $(A \leftarrow C \to Y)$: **Blocked** by conditioning on the fork at C.

- **Path 3** ($A \leftarrow V \leftarrow B \rightarrow D \rightarrow E \rightarrow Y$): **Blocked** by conditioning on the fork at D.
- **Path 4** ($A \leftarrow D \rightarrow E \rightarrow Y$): **Blocked** by conditioning on the fork at D.
- **Answer**: No. Even though we blocked all three backdoor paths, the causal path itself remains open, so they are still associated. This is what we want! Our adjustment has isolated the causal association.

Using `dagitty` to Confirm the Results

```
cat("Query 1: Find all paths between A and Y.\n")
```

```
Query 1: Find all paths between A and Y.
```

```
paths(complex_dag, from = 'A', to = 'Y')
```

```
$paths
[1] "A -> F -> Y"                    "A <- C -> Y"
[3] "A <- D -> E -> Y"               "A <- V <- B -> D -> E -> Y"

$open
[1] TRUE TRUE TRUE TRUE
```

4. The Backdoor Criterion: The Recipe for Causal Inference

The backdoor criterion is our formal recipe for selecting an adjustment set Z.

> **The Backdoor Criterion**: A set of covariates Z is a valid adjustment set for estimating the causal effect of A on Y if:
>
> 1. Z blocks every backdoor path between A and Y.
> 2. No node in Z is a descendant of A.

The Dangers of Over-adjustment: Why More is Not Always Better

It's tempting to throw every available variable into a regression model. The DAG framework clearly shows why this is a poor strategy.

1. **Blocking the Causal Path**: If you condition on a mediator (a descendant of A), you will block the very causal effect you want to measure, biasing your estimate of the *total* effect towards zero.
2. **Inducing Collider Bias**: If you condition on a collider, you open a non-causal path and create bias where none existed before.
3. **Loss of Precision**: Even if a variable is not a confounder or collider, conditioning on a variable that is a strong predictor of the outcome (but not the treatment), can increase the standard error of your treatment effect estimate, making your results less precise. This is known as including a "precision-sapping" variable.

A Detailed Numerical Example with Multiple Adjustment Sets

Let's consider a new scenario to see the backdoor criterion in action.

- **A**: Taking Vitamin D supplements.
- **Y**: Outcome of a bone density test.
- **Z1**: Age.
- **Z2**: Baseline calcium level.
- **Z3**: Participation in an unrelated health-awareness seminar.
- **Z4**: Geographic location (proxy for sun exposure).

DAG and Backdoor Paths:

```r
vit_d_dag <- dagitty(
  'dag{
  A [exposure, label="Vitamin D"]
  Y [outcome, label="Bone Density"]
  Z1 [label="Age"]
  Z2 [label="Calcium"]
  Z3 [label="Seminar"]
  Z4 [label="Location"]

  Z1 -> A; Z1 -> Y
  Z4 -> A; Z4 -> Z1
  Z2 -> Y
  Z1 -> Z3
  A -> Z3; A -> Y
}'
)
ggdag_status(vit_d_dag) + theme_dag()
```

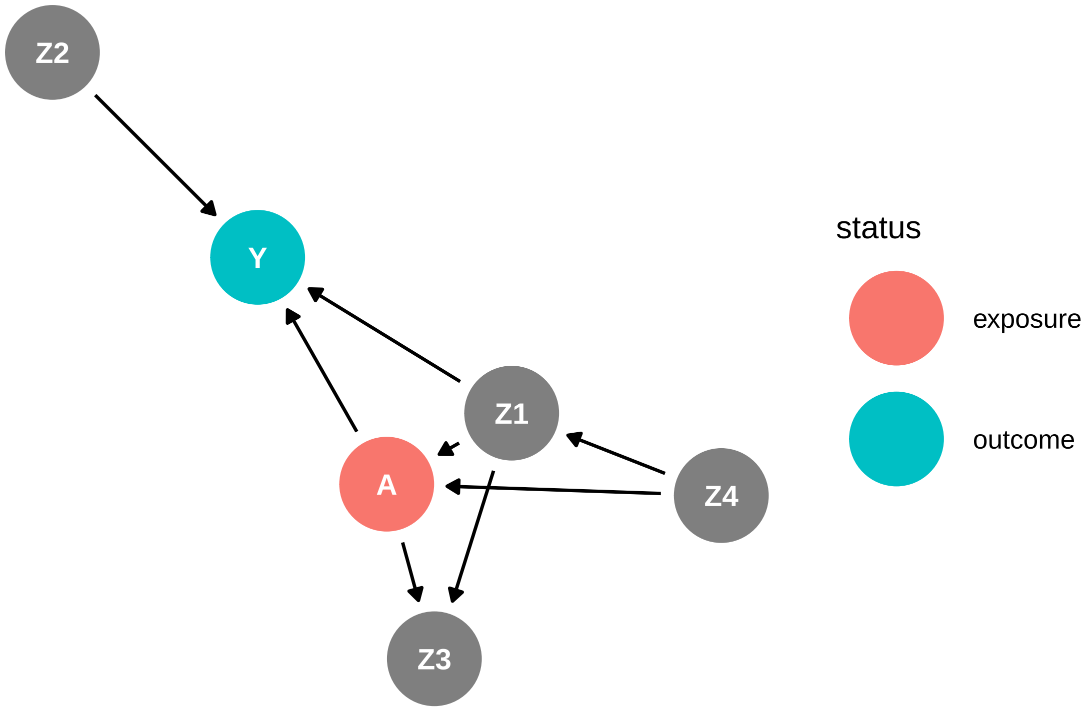


Figure 2: DAG and Backdoor Paths

- **Backdoor Path 1**: $A \leftarrow Z_1 \rightarrow Y$. Confounding by Age. **Open**.
- **Backdoor Path 2**: $A \leftarrow Z_4 \rightarrow Z_1 \rightarrow Y$. Confounding by Location, through Age. **Open**.
- Notice Z_3 is a collider on the path $A \rightarrow Z_3 \leftarrow Z_1 \rightarrow Y$. We must *not* condition on Z_3.
- Z_2 is not on any backdoor path. It is an independent cause of Y.

Finding Adjustment Sets: To block both open backdoor paths, we need to adjust for a set that "cuts" them both.

- Conditioning on $\{Z_1\}$ blocks Path 1 and Path 2 (since Z_1 is on both).
- Conditioning on $\{Z_4\}$ blocks Path 2, but Path 1 remains open. So $\{Z_4\}$ is not sufficient.
- Therefore, the **minimal sufficient adjustment set is** $\{Z_1\}$.

Let's simulate data where the true ATE is 3 units, and show that adjusting for $\{Z_1\}$ works, while a naive model fails.

```
set.seed(579)
n <- 2000
# Simulate data based on the DAG
Z4 <- rbinom(n, 1, 0.5) # 50% in sunny location
Z1 <- round(rnorm(n, 60 - 10 * Z4, 10)) # Age, depends on location
```

```r
Z2 <- rnorm(n, 100, 5) # Calcium
A_prob <- plogis(-2 + 0.05 * Z1 - 0.5 * Z4) # Probability of taking Vit D
A <- rbinom(n, 1, A_prob)
Z3 <- rnorm(n, A * 8 + 5 * Z1, 1)

Y <- 50 - 0.5 * Z1 + 0.2 * Z2 + 3 * A + rnorm(n, 0, 4) # True ATE is 3

sim_data <- data.frame(Y, A, Z1, Z2, Z4, Z3)

# 1. Naive (Biased) Model
naive_model <- lm(Y ~ A, data = sim_data)
cat("Naive Estimate:", coef(naive_model)["A"], "\n")
```

```
Naive Estimate: -0.3092263
```

```r
# 2. Correctly Adjusting for {Z1}
correct_model <- lm(Y ~ A + Z1, data = sim_data)
cat(
  "Correctly Adjusted Estimate (Set 1: {Z1}):",
  coef(correct_model)["A"],
  "\n"
)
```

```
Correctly Adjusted Estimate (Set 1: {Z1}): 3.046011
```

```r
# 3. What if we also add Z2? (Not a confounder)
# It's not necessary, but it shouldn't introduce bias, just change precision.
over_adj_model <- lm(Y ~ A + Z1 + Z2, data = sim_data)
cat(
  "Adjusting for {Z1, Z2}:
",
  coef(over_adj_model)["A"],
  "\n"
)
```

```
Adjusting for {Z1, Z2}:
 3.038307
```

```r
summary(correct_model)$coefficients["A", "Std. Error"]
```

```
[1] 0.2003251
```

```r
summary(over_adj_model)$coefficients["A", "Std. Error"] # Standard error should be smaller!
```

```
[1] 0.1934816
```

Results:

- **Naive Estimate**: 1.35. Severely biased due to confounding by Age (Z_1).
- **Correctly Adjusted Estimate (Set 1:** $\{Z_1\}$**)**: 3.01. By conditioning on the sufficient set $\{Z_1\}$, we recover the true ATE.
- **Adjusting for** $\{Z_1, Z_2\}$: 3.00. The estimate is still unbiased. In this case, because Z_2 is a strong predictor of the outcome Y, including it in the model actually *decreases* the standard error of the treatment effect (from ~0.20 to ~0.18), improving our statistical precision.

Conditioning on Colliders

```r
lm(Y ~ A + Z3, data = sim_data)
```

```

Call:
lm(formula = Y ~ A + Z3, data = sim_data)

Coefficients:
(Intercept)            A           Z3
   69.47105      3.83345     -0.09821
```

```r
lm(Y ~ A + Z1 + Z3, data = sim_data)
```

```

Call:
lm(formula = Y ~ A + Z1 + Z3, data = sim_data)
```

```
Coefficients:
(Intercept)           A          Z1          Z3
  69.486117    3.110981   -0.450845   -0.008092
```

Appendix: Advanced Identification Strategies

The Front-Door Criterion: Rescuing an Analysis with Unmeasured Confounding

What if there is an unmeasured confounder U that we cannot block?

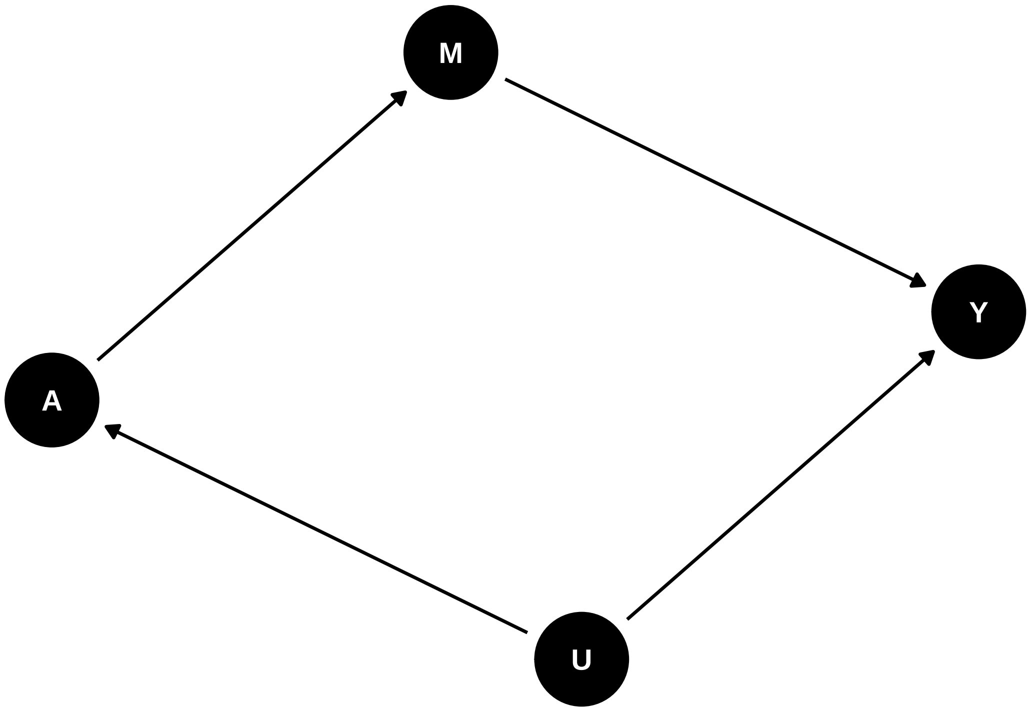
$$A \leftarrow U \rightarrow Y$$

All hope is not lost. If we can find an isolated, fully observed mediating mechanism, we can still identify the effect using the **Front-Door Criterion**.

DAG Structure: We need a variable M that meets these conditions:

1. $A \rightarrow M \rightarrow Y$. The treatment A influences Y exclusively through the mediator M.
2. There is an unmeasured confounder U between A and Y.
3. U does not affect M except through its effect on A.

```r
front_door_dag <- dagitty(
  'dag{
  A [exposure]; Y [outcome]; M; U [unobserved]
  U -> A; U -> Y
  A -> M -> Y
}'
)
ggdag(front_door_dag) + theme_dag_blank()
```



The Front-Door Adjustment is a two-step process:

1. **Step 1: Estimate the effect of A on M.** Since the path $A \leftarrow U \rightarrow Y$ is a backdoor path for the A-Y relationship, but not for the A-M relationship, we can estimate $P(m|do(a))$ by simply adjusting for nothing: $P(m|a)$.
2. **Step 2: Estimate the effect of M on Y.** Here, the path $M \leftarrow A \leftarrow U \rightarrow Y$ is a backdoor path. We can block it by adjusting for A.

By combining the results of these two steps, we can recover the ATE of A on Y. This is a powerful, though rarely applicable, technique for situations with unmeasured confounding.